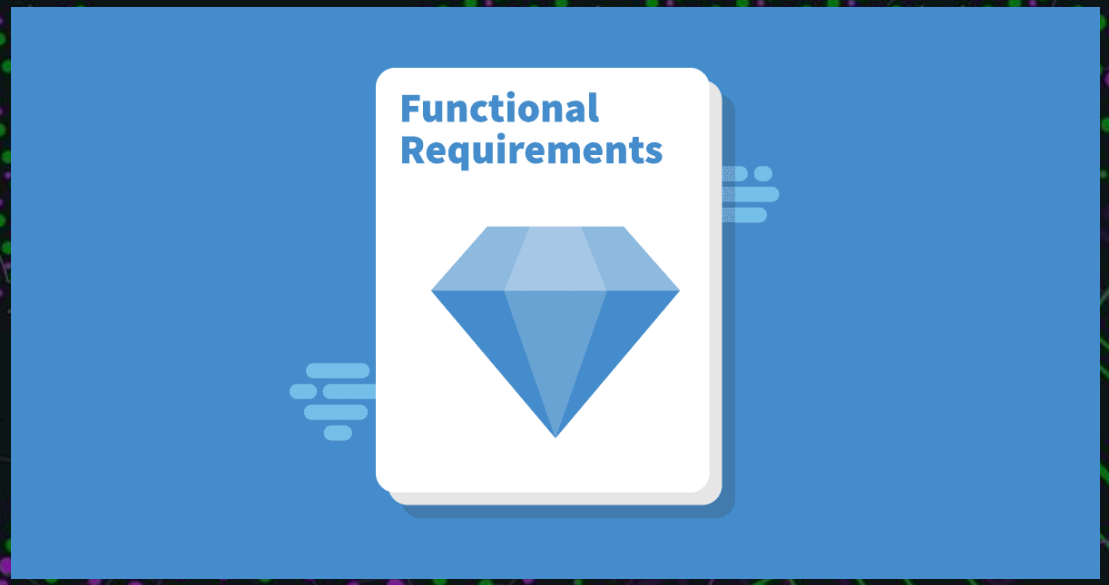
Grouping Functional Requirements

Organizing system functionality logically

In the previous video...



In this video...

Why group functional requirements?

- Define what the application must be able to do/achieve.
- Grouping them helps organize features into logical sections.
- Makes systems easier to understand.
- Reduces complexity by organizing related features together.
- Makes development easier because teams work on different modules.
- Improves clarity for stakeholders and developers.

How to group?

- There are 3 methods:
  1. By feature/module:
     - Group requirements based on system features
  2. By user role:
     - Group based on who performs an action
  3. By process/workflow
     - Based on steps in a process

From previous video:

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Method 1

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Method 1

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Method 1

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Method 1

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Grouped:

- Arithmetic Calculations:
  - Perform basic arithmetic functions:
    - Perform addition
    - Perform subtraction
    - Perform multiplication
    - Perform division

- Input Handling:
  - Accept input
  - Input must be 1 or 2 digits
  - Input must contain valid hexadecimal numbers
  - Input must include a valid operator (+, -, /, *)

- Output Handling:
  - Return and display output
  - Output must be less than 5 digits (<=4)
  - Output must not be negative
  - Output must not contain decimal places

- Error Handling:
  - Invalid hexadecimal input must return an error
  - Missing or invalid operator must return an error
  - Input exceeding 2 digits must return an error
  - Division by zero must return an error
  - Output that is greater than 4 digits will return an error
  - Output that is negative will return an error
  - Output with decimal places will return an error

Thank you for watching